Resport

Part2: **Benchmarking:**

For the small test case, the matrix-vector multiplication uses a 3000 × 3000 matrix, while the matrix-matrix multiplication uses 500 × 500 matrices. For the medium test case, the matrix-vector multiplication uses a 4000 × 4000 matrix, and the matrix-matrix multiplication uses 1000 × 1000 matrices. For the large test case, the matrix-vector multiplication uses a 5000 × 5000 matrix, while the matrix-matrix multiplication uses 1500 × 1500 matrices.

	small			Avg	S.D.	medium			Avg	S.D.	large			Avg	S.D.
	1	2	3			1	2	3			1	2	3		
Matrix-Vector Multiplication (Row-Major)	23	23	25	23.67	1.15	36	51	37	41.33	8.39	67	54	54	58.33	7.51
Matrix-Vector Multiplication (Column-Major)	45	58	45	49.33	7.51	145	158	134	145.67	12.01	179	204	140	174.33	32.25
Matrix-Matrix Multiplication (Naive)	281	261	271	271.00	10.00	2476	2428	2540	2481.33	56.19	13073	12523	12878	12824.67	278.85
Matrix-Matrix Multiplication (Transposed B)	276	271	246	264.33	16.07	1968	2057	2108	2044.33	70.85	7335	7238	7753	7442.00	273.67

Part2: **Cache Locality Analysis**

Matrix*Vector

We can find from the graph above, for matrix-vector multiplication, the row-major implementation is expected to perform better, because the inner loop accesses matrix elements contiguously in memory. In contrast, the column-major implementation accesses elements with a stride of rows, which reduces spatial locality and leads to poorer cache utilization.

Matrix*Matrix

In the naive implementation, matrix B is accessed column by column while stored in row-major order, which causes strided memory access. In the transposed-B implementation, B is first transposed so that the inner loop accesses contiguous memory from both A and B^T, improving cache utilization.

Stride Benchmark

In addition, I ran a stride benchmark on a large 1D array with strides 1, 2, 4, 8, 16, 32, and 64 to isolate the impact of cache locality on runtime to test the stride benchmark. In this stride benchmark, the total runtime generally decreases as the stride increases, mainly because fewer elements are accessed. Therefore, the raw runtime does not imply that larger strides have better cache behavior. From a cache-locality perspective, small strides, especially stride 1, usually achieve better spatial locality because more elements within each cache line are used. Larger strides tend to waste more of each fetched cache line.

```
=== Stride Benchmark ===
Stride 1: 33.7267 ms
Stride 2: 21.5662 ms
Stride 4: 9.15444 ms
Stride 8: 6.23266 ms
Stride 16: 6.70026 ms
Stride 32: 7.40074 ms
Stride 64: 3.8695 ms
PS D:\Uchicago Courses\2026spring\Cfin\Hi
```

Part2: **Memory Alignment:**

The original implementation uses standard dynamic allocation for matrices, vectors, and result buffers. An aligned version was created by allocating all arrays on 64-byte boundaries and benchmarking the same four kernels: row-major matrix-vector multiplication, column-major matrix-vector multiplication, naive matrix-matrix multiplication, and matrix-matrix multiplication with transposed B. The same dimensions and computation logic were used in both versions. Baseline code allocates matrices and result buffers with ordinary dynamic allocation and benchmarks the four kernels in main().

The results show that cache locality matters more than alignment. In matrix-vector multiplication, row-major was much faster than column-major because its inner loop accesses contiguous memory. In matrix-matrix multiplication, the transposed-B version was much faster than the naive version because it converts strided accesses to contiguous accesses. Memory alignment helped noticeably for matrix-vector multiplication, but its effect on matrix-matrix multiplication was limited and inconsistent. Therefore, the main source of performance improvement in this project came from improving memory access patterns rather than alignment alone.

```
Time_row_major_Matrix_vector_multiplication: 57 milliseconds
Time_col_major_Matrix_vector_multiplication: 174 milliseconds
Time_col_major_Matrix_Matrix_multiplication: 12412 milliseconds
Time_col_major_Matrix_Matrix_multiplication_transposed: 6980 milliseconds
```

```
Stride 64: 3.1296 ms
Aligned row-major MV: 6 ms
Aligned col-major MV: 11 ms
Aligned naive MM: 11745 ms
Aligned transposed-B MM: 7281 ms
```

Part2: **Inlining**

To study the effect of inlining, I introduced a very small helper function for row-major matrix access and created two versions of the same matrix-vector multiplication kernel. The first version called a normal helper function, while the second version called an inline helper function. The goal was to isolate the effect of function-call overhead inside the inner loop of the matrix-vector multiplication.

The benchmark used the same matrix size, input vector, and result buffer structure for both versions, so the only intended difference was whether the helper function was explicitly marked as inline. Both versions were executed multiple times, and the average runtime was reported.

This result suggests that inlining provided a better performance benefit in this experiment.

```
=== Inlining Benchmark ===
MV helper (no inline): 69.3667 ms
MV helper (inline): 65.4841 ms
PS D:\Uchicago Courses\2026spring\Cfin\High-Performance-Line
```

Part2: **Profiling:**

My computer cannot run Visual Studio and WPT. I don't know the reason, I will try to fix it soon.

Part2: **Optimization Strategies**

I may chose to optimize the baseline naive matrix-matrix multiplication using **loop reordering**, because the profiling and cache-locality analysis showed that the main bottleneck was in the innermost loop of multiply_mm_naive, especially the access to matrix **B**. In the original version, the loop order is i-j-k, and the code accesses matrixB[k * colsB + j]. This means that for a fixed j, the inner loop moves down a column of B, which is a strided access pattern in row-major storage and leads to poor cache locality. To improve this, I changed the loop order from i-j-k to i-k-j. In the reordered version, I first load A[i][k] once into a temporary variable, and then iterate over j, updating the whole row of the result with result[i * colsB + j] += aik * matrixB[k * colsB + j]. This makes the access to matrixB contiguous in memory because j now changes in the innermost loop, which matches row-major layout much better.

I think this optimization is effective because it improves cache locality and cache reuse. In the

original naive version, elements of B are accessed with a large stride, so each cache line is used inefficiently. In the reordered version, the program scans each row of B sequentially, which is much more cache-friendly. At the same time, each A[i][k] value is reused across the entire inner j loop, reducing repeated loads. Therefore, loop reordering improves memory access efficiency without changing the mathematical result, making it a simple but meaningful optimization for the baseline matrix multiplication.